

Selecting What to Remember: Validation-Gated Memory for LLM Review of Reinforcement Learning Trading Decisions

Nidhish Gautam

Department of Electrical and
Computer Engineering
New York University, Tandon School
of Engineering
Brooklyn, New York, USA
ng3483@nyu.edu

Quanyan Zhu

Department of Electrical and
Computer Engineering
New York University, Tandon School
of Engineering
Brooklyn, New York, USA
qz494@nyu.edu

Mirela M. Ivan

Department of Finance and Risk
Engineering
New York University, Tandon School
of Engineering
Brooklyn, New York, USA
mivan@nyu.edu

Code: <https://github.com/nidhish1/validated-memory-rl-trading>

Abstract

We ask whether an already-trained reinforcement learning trading system can be improved by placing a frozen large language model (LLM) reviewer in front of its orders, without retraining any component and under a small API budget. The base system pairs a multilayer perceptron (MLP), which classifies the next market move as bearish, neutral, or bullish, with an ensemble of Proximal Policy Optimization (PPO) agents, which propose a continuous exposure; a fixed weighted sum of the two becomes the daily trading proposal. The reviewer sees the market features, the component outputs, the portfolio state, and the cost of each permitted action, and may only keep, halve, or flatten that proposal. Because an LLM does not learn from the outcomes of its own past decisions, we add a memory layer: over a development period we pair each review with its realized outcome, let the LLM reflect on these records to draft reusable review instructions, and then screen the drafts and replay each survivor on a later, disjoint validation period. Only instructions that raise the validation Sharpe ratio without materially increasing drawdown are retained, and the retained memory is frozen for the final evaluation. On daily Ethereum data, with 20M tokens costing merely \$4 on DeepSeek Flash V4, a single gate-selected instruction raised the final-period Sharpe ratio from 5.638 to 5.763 (+2.2%) and net return from 10,009% to 10,763%, at an unchanged maximum drawdown of 18.59%. Memory that was not validated hurt: the reviewer with all 20 generated instructions, with the 5 screened instructions, or with no memory at all underperformed the base system by 7–11% in Sharpe ratio. These results come from one asset and one market path and are preliminary.

CCS Concepts

• **Computing methodologies** → **Reinforcement learning**; *Natural language processing*; • **Applied computing** → *Economics*.

Keywords

reinforcement learning, large language models, algorithmic trading, memory, experience-based learning, LLM agents, context engineering

1 Introduction

Automated trading systems are expensive to retrain and risky to modify once deployed. A system that performs acceptably in production therefore tends to stay as it is, even when its operators

can describe, in words, the situations in which it behaves poorly. This paper asks whether that verbal knowledge can be put to work without touching the deployed models: can a frozen LLM, inserted as a reviewer between a trained trading policy and the order it would send, improve risk-adjusted performance, and can such a reviewer accumulate usable experience when its own weights never change?

The system is primarily built for automated daily trading. The base system combines a multilayer perceptron (MLP), which predicts the market direction, with a reinforcement learning agent trained using Proximal Policy Optimization (PPO) [15], which is responsible for proposing the direction and size of the trade.

Finally, the PPO proposals are averaged, and the MLP’s signed trading signal and the averaged PPO proposal are combined into a weighted sum. This becomes our base model. Figure 1 shows the base system and the proposed memory layer.

The intuition behind adding a reviewer is that an LLM can use the knowledge from its training to interpret the information provided and identify possible reasons to adjust the proposal. For example, a strong PPO proposal may be accompanied by a conflicting MLP prediction. The LLM reviewer can consider this disagreement together with volatility, the existing position, and the cost of changing the exposure. An LLM does not automatically learn from the outcomes of its previous decisions. We therefore built a memory layer that stores lessons from earlier reviews and supplies them as instructions in later decisions. The idea is to turn previous experience into a reusable procedure while keeping the LLM’s weights unchanged.

Not every lesson generated from experience will improve the system, and a losing trade does not necessarily mean that the decision was wrong. We therefore screen the candidate lessons and test the surviving ones on a separate validation period before deciding which memory to retain. The selected memory is then kept fixed during the final evaluation.

This paper makes two contributions.

- We describe a review layer that leaves a trained MLP–PPO trader untouched and restricts the LLM to three cost-aware actions on an existing proposal (KEEP, HALVE, FLAT), so that any performance change is attributable to the review decision alone.
- We separate memory *generation* from memory *retention* and make retention an explicit, chronologically disjoint validation gate with stated Sharpe, drawdown, and growth conditions.

2 Related Work

Memory and self-improvement in LLM agents. A line of work lets a language model improve without weight updates by writing text it later reads back. Self-Refine [11] revises a single output through self-feedback; Reflexion [17] keeps verbal reflections across attempts; ExpeL [26] extracts transferable insights from a pool of past trajectories; Voyager [18] accumulates a reusable skill library; generative agents [14] rank stored observations by recency and importance; and MemGPT [13] manages what stays in a bounded context.

Automatic optimization of prompts and context. Instructions can also be searched over directly. APE [27] and OPRO [20] use a language model to propose and score candidate prompts, DSPy [7] compiles and optimizes multi-stage pipelines, GEPA [1] evolves prompts by reflecting on execution traces, and ACE [23] curates structured playbooks. We adopt the propose-then-select principle of this literature but apply it to a sequential trading account, where a retained instruction alters the positions that all later decisions inherit, so a candidate must be judged on a realized equity path rather than on per-example accuracy.

Language models in financial decision-making. Lopez-Lira and Tang [10] report that LLM news sentiment carries return-predictive signal, and a number of agent systems act on such signals directly: FinMem [21] equips a trading agent with layered memory, FinAgent [24] adds tool use and multimodal inputs, TradingAgents [19] and FinCon [22] assign roles to multiple cooperating agents, and Darmanin and Vella [3] feed LLM guidance to a DDQN agent as an extra observation. In all of these the language model helps form the trading view. In ours it never does: the MLP-PPO ensemble produces the proposal, and the reviewer may only keep, halve, or flatten it, which isolates the contribution of the review decision itself.

Reinforcement learning for trading. Our base system follows the direct-reinforcement tradition of Moody and Saffell [12], who optimize a differential Sharpe ratio rather than per-trade profit, as extended by Deng et al. [4] with learned representations and by Zhang et al. [25] and FinRL [8] to modern deep policies trained with algorithms such as PPO [15]. We take such a trained system as given and leave its weights untouched.

We focus on whether experience-driven lessons improve trading performance and how the decision to retain those lessons affects the results. Candidate generation, assisted screening, and performance validation are separate, and the selected memory remains fixed during the final evaluation. This design allows us to compare different memory-retention strategies using the same base proposals and replay setting. Relative to the work above, our emphasis is less on how to produce good reflections and more on how to decide which reflections deserve to be kept.

3 Methodology

Our system has three parts: the base exposure proposal (MLP + PPO), the LLM reviewer, which decides how much of the proposal to follow, and the memory layer. The memory is built from earlier reviews, candidate lessons are tested on a separate period, and the selected memory is kept fixed for the final evaluation.

3.1 Base Trading Model

The base model combines an MLP classifier with three PPO exposure policies. We index trading days by t , and write $\tilde{x}_t \in \mathbb{R}^{54}$ for the standardized feature vector of day t , which includes return, momentum, volatility, and aggregated sentiment features. We use $\tilde{x}_t$ for standardized features and $\tilde{a}_t$ for the sampled action before clipping. Standardization uses parameters estimated from the original training period, which remain fixed afterward. The MLP predicts bearish, neutral, and bullish market classes using labels constructed through the triple-barrier procedure [9]:

$$\mathbf{q}_t = \text{softmax}(f_\phi(\tilde{x}_t)), \quad m_t = d_t q_t(d_t),$$

where f_ϕ is the MLP, with parameters ϕ , $\mathbf{q}_t$ contains its class probabilities and $q_t(c)$ denotes the entry of $\mathbf{q}_t$ for class c , with $c = -1, 0$, and $+1$ denoting bearish, neutral, and bullish, and $d_t = \arg \max_{c \in \{-1, 0, 1\}} q_t(c)$. The signed signal m_t combines the predicted direction with its class probability, so a confident bullish call contributes a value near $+1$, a confident bearish call a value near -1 , and a neutral call exactly zero.

During training, each PPO agent observes standardized features, previous exposure, and the most recent net trading return. Its policy is written as

$$o_t = \begin{bmatrix} \tilde{x}_t \\ a_{t-1} \\ R_t \end{bmatrix}, \quad \tilde{a}_t \sim \pi_\theta(\cdot | o_t), \quad a_t = \text{clip}(\tilde{a}_t, -1, 1).$$

Here, o_t is the agent's observation, π_θ is the policy parameterized by θ , $\tilde{a}_t$ is the sampled action, and a_t is the exposure applied by the training environment. The entry R_t of o_t is the net return of the previous step, defined in the next equation. The resulting net return is

$$R_{t+1} = a_t r_{t+1} - \kappa |a_t - a_{t-1}|,$$

where r_{t+1} is the next daily market return and κ is the fee plus slippage rate, so the second term charges the agent for the notional it turns over. This return enters the differential Sharpe reward calculation [12] and becomes part of the next observation. PPO uses the collected experience to update the policy parameters through its clipped policy objective.

We train three agents with different random seeds and approximately 200,000 environment steps per agent. After training, their deterministic exposure proposals are averaged and combined with the MLP signal:

$$b_t = \alpha m_t + (1 - \alpha) \frac{1}{3} \sum_{k=1}^3 a_t^{(k)},$$

where $b_t \in [-1, 1]$ is the *base exposure proposal*, $a_t^{(k)}$ is the deterministic proposal from PPO agent k for $k \in \{1, 2, 3\}$, and $\alpha \approx 0.2792$. The weight α was fitted during the original development of the base system and is treated here as a fixed constant.

During the experiment, the base model and ensemble weights remain fixed. All reviewer conditions receive the same reconstructed proposal sequence. For proposal reconstruction, each agent receives its own previous deterministic proposal and a recent-return input

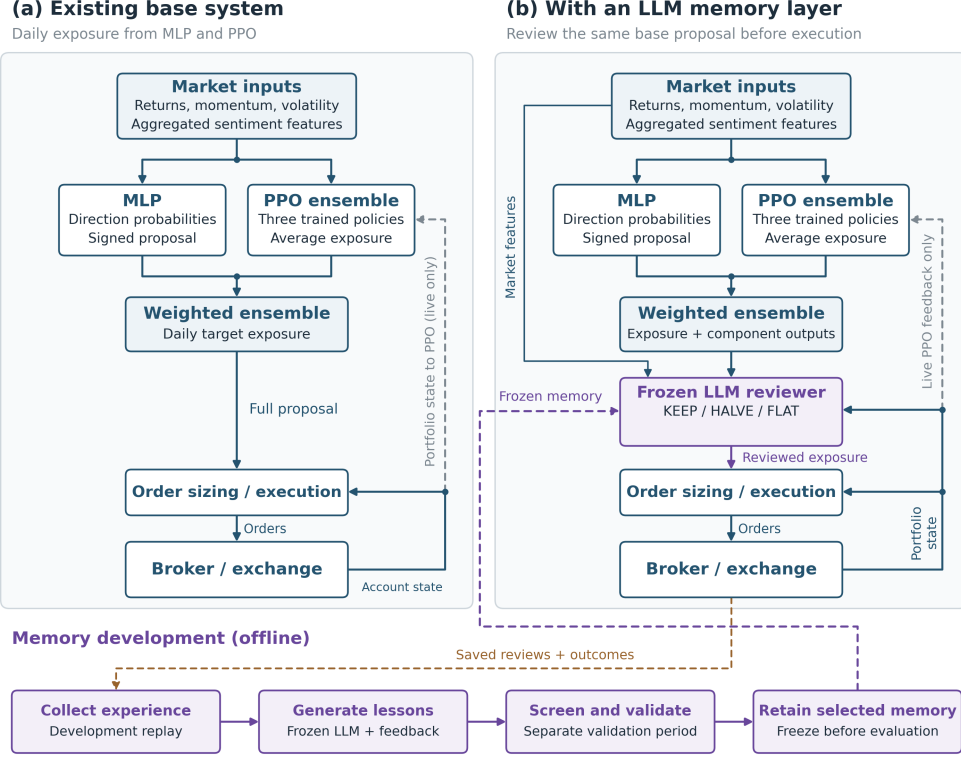


Figure 1: The diagram compares (a) the existing MLP–PPO trading system with (b) the proposed LLM memory layer and summarizes the offline memory-development process.

fixed at zero,

$$o_t^{(k), \text{replay}} = \begin{bmatrix} \tilde{x}_t \\ a_{t-1}^{(k)} \\ 0 \end{bmatrix}, \quad a_{a-1}^{(k)} = 0,$$

initialized flat at the first decision of each replay period. These inputs are independent of the reviewer’s account state, which is what makes the proposal sequence common to all conditions. Note that the training observation of Section 3.1 carries the previous realized net return, whereas the reconstruction fixes that coordinate at zero. Each reviewer maintains its own portfolio state separately. Holding b_t identical across conditions is what makes the comparison in Section 5 a comparison of review decisions rather than of trading signals.

3.2 LLM Reviewer and Memory Layer

The reviewer receives the outputs of the base model at each trading decision, along with market features, portfolio state, and the transaction costs associated with each permitted action. We represent this as

$$s_t = (z_t, f_t, p_t, C_t),$$

where s_t is the *reviewer state* and z_t contains the ensemble, MLP, and PPO outputs b_t , m_t , and $a_t^{(1)}, a_t^{(2)}, a_t^{(3)}$; f_t contains the market features (a human-readable subset of $\tilde{x}_t$); p_t contains the reviewer’s portfolio state; and C_t contains the computed action costs of the

three permitted actions. Writing u_{t-1}^M for the exposure the reviewer currently holds, these are

$$c_t^M(h) = \kappa |\lambda(h)b_t - u_{t-1}^M|,$$

$$C_t^M = (c_t^M(\text{KEEP}), c_t^M(\text{HALVE}), c_t^M(\text{FLAT})),$$

expressed as fractions of account equity and computed from the reviewer’s own current exposure, consistently with Definition 3.2. Note that KEEP does not mean no trade, since following a changed base proposal still incurs turnover, and FLAT incurs a liquidation cost whenever the account is not already flat.

Given the instructions I and memory M , the frozen LLM produces an action h_t and an explanation e_t :

$$(h_t, e_t) = F_\psi(I, s_t, M),$$

where F_ψ is the frozen LLM and ψ denotes its parameters, which remain unchanged throughout the experiment. We write ψ rather than θ because θ already denotes the PPO policy parameters, and the two models are distinct and separately parameterized. The action determines how much of the base proposal to follow:

$$u_t = \lambda(h_t)b_t, \quad \lambda(h_t) = \begin{cases} 1, & h_t = \text{KEEP}, \\ \frac{1}{2}, & h_t = \text{HALVE}, \\ 0, & h_t = \text{FLAT}. \end{cases}$$

Here, b_t is the base proposal, $\lambda(h_t)$ is the fraction of it retained, and u_t is the *target exposure* the account actually takes on day t ; the

mean absolute target exposure reported in Section 5 is the average of $|u_t|$ over the evaluation period. KEEP follows the full proposal, HALVE reduces it by half, and FLAT sets the exposure to zero. The action set is deliberately one-sided: the reviewer can scale exposure down but never up, and cannot reverse the sign of the proposal. Any improvement must therefore come from declining risk on the right days, not from taking a different view of the market.

3.3 Replay and Performance Measures

Throughout, fix a period $\mathcal{T}$ with $n = |\mathcal{T}|$ decisions and the common base proposal sequence $b = (b_t)_{t \in \mathcal{T}}$.

Definition 3.1 (Induced exposure path). For a memory M , write h_t^M for the action the reviewer returns at day t and $\lambda_t^M = \lambda(h_t^M) \in \{0, \frac{1}{2}, 1\}$ for the corresponding retention factor. The *induced exposure path* is $u^M = (u_t^M)_{t \in \mathcal{T}}$ with $u_t^M = \lambda_t^M b_t$. The base account is the special case $\lambda_t \equiv 1$ and the fixed-half control $\lambda_t \equiv \frac{1}{2}$.

Because the reviewer state s_t contains the portfolio state $\mathbf{p}_t$, which depends on the exposures already taken, λ_t^M is a function of the entire history up to t and not of t alone.

Definition 3.2 (Performance functionals). Let $\mathcal{T} = \{a, \dots, b\}$ with $n = b - a + 1 \geq 2$. Each replay starts flat and solvent, $u_{a-1} = 0$ and $W_a > 0$, and we assume every path is *admissible*, in that $R_{t+1}(u) > -1$ for each decision. For $t \in \{a, \dots, b-1\}$ the realized net return is

$$R_{t+1}(u) = u_t r_{t+1} - \kappa |u_t - u_{t-1}|,$$

and on the final decision the remaining position is liquidated at the same cost rate, so

$$R_{b+1}(u) = u_b r_{b+1} - \kappa |u_b - u_{b-1}| - \kappa |u_b|.$$

Equity evolves as $W_{t+1} = W_t(1 + R_{t+1}(u))$. We write

$$G(u) = \sum_{t \in \mathcal{T}} \log(1 + R_{t+1}(u))$$

for total log growth,

$$D(u) = \max_{a \leq t \leq b+1} \left(1 - W_t / \max_{a \leq s \leq t} W_s \right)$$

for maximum drawdown, which runs to W_{b+1} so that the final outcome is included, and

$$S(u) = \sqrt{365} \frac{\bar{R}(u) - r_f}{\hat{\sigma}(u)}$$

for the annualized Sharpe ratio, where $\bar{R}$ and $\hat{\sigma}$ are the sample mean and sample standard deviation of the n returns $\{R_{t+1}(u)\}_{t \in \mathcal{T}}$, assumed to satisfy $\hat{\sigma}(u) > 0$, and r_f is the daily risk-free rate. The terminal liquidation charge therefore enters equity, G , D , and the return series underlying S through the single final element R_{b+1} . The quantities S_i , D_i , and G_i of Section 3.5 are $S(u^{\{\ell_i\}})$, $D(u^{\{\ell_i\}})$, and $G(u^{\{\ell_i\}})$ evaluated on $\mathcal{T}_{\text{val}}$.

3.4 Experience and Candidate Memory

Let $\mathcal{T}_{\text{build}}$, $\mathcal{T}_{\text{val}}$, and $\mathcal{T}_{\text{eval}}$ denote the sets of day indices in the memory-development, memory-validation, and final-evaluation periods of Table 3; they are chronologically ordered and pairwise disjoint. We collect the development experience with empty memory, $M = \emptyset$. Each saved case contains the reviewer's state, action, explanation,

and subsequent outcome, recorded as a chronologically ordered sequence rather than a set, since reflection consumes it in order:

$$\mathcal{D}_{\text{build}} = \left((s_t, h_t, e_t, R_{t+1}, \mathbf{R}_{t+1}^{\text{alt}}) \right)_{t \in \mathcal{T}_{\text{build}}},$$

where R_{t+1} is the realized net replay return, computed as in the net-return equation above but with the reviewer's target exposure u_t in place of a_t , and $\mathbf{R}_{t+1}^{\text{alt}}$ contains the one-step returns for KEEP, HALVE, and FLAT from the same account state, namely

$$R_{t+1}^{\text{alt}, M}(h) = \lambda(h) b_t r_{t+1} - \kappa |\lambda(h) b_t - u_{t-1}^M|.$$

These are one-step counterfactual returns conditional on the observed account state; they do not simulate the subsequent portfolio trajectory that would have followed an alternative action. These outcomes are supplied as feedback after the trading outcome is observed. Including the counterfactual returns lets the reflection step see what the two actions not taken would have earned from the same account state, which a realized-return-only record cannot show.

The frozen LLM examines chronological batches of these experiences and may propose a reusable instruction for each batch. Assisted screening then identifies the candidates suitable for validation:

$$C = \text{Reflect}_{\psi}(\mathcal{D}_{\text{build}}),$$

$$C_{\text{screen}} = \text{Screen}_{\text{assisted}}(C, \mathcal{D}_{\text{build}}).$$

Here C is the set of generated candidate lessons and $C_{\text{screen}} \subseteq C$ those admitted to validation. No candidate is activated during the initial experience collection, so the development trajectory is generated with empty memory throughout.

For our experiment, we chronologically divided the 273 development days into 20 batches: 19 full batches of 14 and one half-sized batch of 7. The frozen LLM reviewed each batch and generated one candidate lesson, resulting in 20 candidate memory lessons for screening. We then performed an offline Codex-assisted review of each candidate lesson and its supporting development records. We checked whether its claims matched the model outputs, portfolio position, supplied action costs, and previously saved reviewer explanations. The screening decisions and their reasons were then recorded in the notebook.

For example, we excluded a lesson that claimed HALVE would always reduce turnover and entry cost, because the cost depends on the current position and proposed target. We also excluded lessons that treated an MLP probability near 0.5 as evidence of disagreement between models, because confidence and directional agreement are different. For overlapping lessons about comparing the current position, target exposure, and action costs, we retained the earliest representative for validation. Candidates with unclear wording or insufficient supporting evidence were deferred from validation. The screening outcome counts for all 20 candidates are listed in Table 1.

3.5 Selecting Which Memory to Retain

Each screened lesson $\ell_i \in C_{\text{screen}}$ is evaluated separately by setting $M = \{\ell_i\}$ and replaying the validation period. Each reviewer maintains its own portfolio state while receiving the same base proposals.

Table 1: Outcomes of the offline Codex-assisted screening step.

Screening outcome	Lessons
Admitted to validation	5
Excluded: semantic issues	8
Excluded: redundant	3
Deferred	4

Note: Screening was an interactive AI-assisted research step. The notebook recorded the decisions and their reasons but did not independently perform the screening through an automated procedure. Automating this process and evaluating its consistency and reliability are directions for future work.

Let S_i , D_i , and G_i represent the Sharpe ratio, maximum drawdown, and total log growth with candidate i . The corresponding values without memory are S_0 , D_0 , and G_0 . A lesson passes validation only if

$$\begin{aligned} S_i &\geq S_0 + 0.10, \\ D_i &\leq D_0 + 0.01, \\ G_i &\geq \begin{cases} 0.90G_0, & G_0 > 0, \\ G_0, & G_0 \leq 0. \end{cases} \end{aligned}$$

These conditions require an improvement in Sharpe while limiting additional drawdown and lost growth. They are selection criteria rather than statistical significance tests. The selection rule and its margins were fixed before the validation replay. The numerical thresholds were then calculated from the no-memory reviewer’s validation results and were not tuned afterward; 0.10 is an absolute increment in the annualized Sharpe ratio and 0.01 is one percentage point of maximum drawdown. Note that S_0 , D_0 , and G_0 are those of the no-memory reviewer, so the gate tests improvement over the no-memory reviewer and does not require improvement over the unreviewed base system.

Let $\mathcal{A}$ contain the indices of the candidates that pass. We retain the lesson with the highest validation Sharpe:

$$M^* = \begin{cases} \{\ell_{i^*}\}, & i^* = \arg \max_{i \in \mathcal{A}} S_i, \quad \mathcal{A} \neq \emptyset, \\ \emptyset, & \mathcal{A} = \emptyset. \end{cases}$$

Exact ties are resolved by lower drawdown and then by candidate identifier. At most one lesson is retained, with its wording unchanged. If no lesson passes, the memory remains empty.

A candidate had to satisfy all three requirements in Table 2.

Table 2: Validation requirements derived from the no-memory reviewer.

Requirement	Calculation	Required value
Sharpe	$6.675763 + 0.10$	≥ 6.775763
Maximum drawdown	$8.531821\% + 1 \text{ percentage point}$	$\leq 9.531821\%$
Total log growth	1.479214×0.90	≥ 1.331293

B013 asks the reviewer to check how the individual models support the ensemble proposal, there might be disagreements between the combined proposal, for example: the PPO agents may support a long position while the MLP predicts a bearish outcome and the three PPO agents might disagree with other. The instruction asks the reviewer to consider these differences alongside proposal magnitude and the MLP’s maximum class probability before deciding

how much exposure to follow. This is a qualitative review instruction. It does not specify exact thresholds for “modest,” “large,” or “near 0.5,” or require HALVE whenever models disagree. A class probability near 0.5 does not itself establish disagreement or represent the probability of trading profit. The reviewer also considers market conditions, its current portfolio position, and the supplied transaction costs. The instruction remained unchanged throughout final evaluation.

3.6 Final Evaluation with Frozen Memory

During the final evaluation, the reviewer uses the selected memory:

$$(h_t, e_t) = F_\psi(I, s_t, M^*), \quad M_t = M^* \quad \text{for all } t \in \mathcal{T}_{\text{eval}},$$

where M_t denotes the memory supplied on day t , so the second equality states that it never changes during evaluation. The market features and portfolio state change at each decision, but the LLM parameters ψ , shared instructions, and selected memory remain fixed. The final outcomes update the account but are not used to revise the memory.

4 Experimental Setup

The experiment is a feasibility study rather than a full evaluation. It uses a single asset, a single market path, one development period, and one validation period, with API spending for memory generation capped at \$10.

4.1 Data and Experimental Periods

We evaluate the memory layer using the exported Ethereum dataset and trained MLP–PPO models. The export contains 3,243 daily feature rows from November 9, 2017, to September 25, 2026. The original training period ends on December 31, 2022, and the original validation period runs from January 1 to September 30, 2023. The existing PPO checkpoints were fitted using both periods. We use the subsequent data for memory development, memory validation, and final evaluation, so no decision evaluated in this paper falls inside the data used to fit the base model.

Table 3 lists the dates and number of daily decisions used for memory development, memory validation, and final evaluation.

Table 3: Memory-development and evaluation periods.

Period	Dates	Daily decisions
Memory development	October 1, 2023–June 29, 2024	273
Memory validation	July 1–December 30, 2024	183
Final evaluation	January 1, 2025–September 24, 2026	632

June 30 and December 31, 2024, are excluded as decision rows to provide buffers between stages. The September 25, 2026, decision is excluded because the price on the exact next calendar day is missing. We do not replace this missing daily outcome with a two-day return.

4.2 Comparison Conditions

The fixed-half control matters because every reviewer action other than KEEP reduces exposure; without it, a performance change caused by de-risking on average could be mistaken for a change caused by de-risking selectively.

Before final evaluation, the validation gate runs ten accounts: two scripted controls, three LLM controls, and five single-candidate reviewers. Candidate selection follows the rule described in the methodology after all ten accounts finish.

Table 4 summarizes the six original comparison conditions and two additional rule-based benchmarks. The additional benchmarks were introduced after inspecting the original final-evaluation results and are therefore treated as exploratory comparisons.

Table 4: Comparison conditions used in the final evaluation.

Condition	Memories	Description
Base model: full exposure	—	Always chooses KEEP, following the full base proposal.
Base model: half exposure	—	Always chooses HALVE, following half of the base proposal.
LLM without memory	0	Reviews each proposal without stored lessons.
LLM with raw memory	20	Receives all generated candidate lessons, without screening or performance-based selection.
LLM with screened memory	5	Receives all five lessons admitted through the assisted screening, without selecting among them using validation performance.
LLM with gate-selected memory	1	Receives B013, selected through the separate validation gate. Its instruction remains fixed during final evaluation.
Model-disagreement rule [*]	—	Chooses HALVE when the MLP direction and averaged PPO proposal have opposite signs; otherwise KEEP.
High-volatility rule [*]	—	Chooses HALVE when 30-day daily-return volatility exceeds the fixed development-period threshold of approximately 3.506%; otherwise KEEP.

^{*}Exploratory benchmarks added after inspecting the original final-evaluation results.

The volatility threshold was calculated as the 75th percentile of positive 30-day volatility values during the memory-development period. It was approximately 0.035055, or 3.506% daily volatility, and remained fixed during validation and final evaluation. Both additional rules use the same proposal sequence and replay settings as the original conditions.

Throughout these comparisons, KEEP means following the full new base proposal, while HALVE means following half of that proposal. Neither action refers to retaining or halving the existing portfolio position.

4.3 LLM and Replay Settings

The reviewer uses the DeepSeek API model identifier `deepseek-flash`, with thinking disabled, temperature set to zero, and a maximum completion length of 512 tokens per review. The same shared instructions, action definitions, and cost inputs are used across LLM conditions. Memory is supplied as additional context, with no LLM weight updates. Memory generation and all replays consumed approximately 20 million tokens at a cost of approximately \$4, against the \$10 cap.

Each account starts with USD 100,000 and no position. Decisions are evaluated daily using the source row’s close and the next calendar day’s close. The replay charges a fee of 0.10% and slippage of 0.05% on traded notional. Any remaining position is closed at the end of each replay period, with the same cost rate applied to liquidation.

Funding costs, borrowing costs, and interest on uninvested cash are not modeled. The replay also does not include the original strategy’s take-profit and stop-loss mechanics.

Accounts run in parallel, while decisions remain chronological within each account. Requests, responses, and daily account

states are saved so interrupted runs can resume. Citation-validation amendments are logged, and the original reviewer decisions are retained.

4.4 Evaluation Measures

We report annualized Sharpe [16], total net return, maximum draw-down, mean absolute target exposure, and the frequency of each reviewer action. Sharpe uses 365-day annualization, sample standard deviation, and a daily risk-free benchmark corresponding to 4% annually. Net returns include trading costs and terminal closing costs. API charges are accounted for separately.

The validation period is used for memory selection, while the final period compares the frozen conditions. All conditions share the same market path, so their account trajectories are not independent market samples. We report point estimates; statistical significance has not been established.

5 Results

Table 5 reports the validation performance and selection outcome of each screened candidate. B013 and B018 satisfied all three requirements. B013 was selected because its validation Sharpe was higher: 7.107 compared with 6.788.

Table 5: Validation performance and selection outcomes for the five screened lessons.

Cand.	Sharpe	Max. DD (%)	Log growth	Selection outcome
B001	7.079	10.092	1.671450	Failed drawdown requirement
B008	6.735	6.755	0.862389	Failed Sharpe and growth requirements
B013	7.107	9.176	1.675782	Passed; selected
B018	6.788	9.141	1.567001	Passed; not selected
B019	6.901	5.465	0.872605	Failed growth requirement

Table 6 reports the final-period results, and Table 7 reports how often each reviewer intervened.

Only validated memory beat the base system. The reviewer carrying the single gate-selected instruction reached a Sharpe ratio of 5.763 against 5.638 for the full-exposure base model, an improvement of 2.2%, while net return rose from 10,009.31% to 10,762.61% and maximum drawdown was unchanged at 18.59%.

Intervention was always partial. In all four LLM conditions the KEEP and HALVE counts account for all 632 decisions, so FLAT was never chosen.

6 Discussion

The two intermediate conditions are the informative ones. Raw memory supplied every lesson the reflection step produced, and screened memory supplied the subset that an assisted screen judged plausible; both pushed the reviewer toward near-constant halving and both underperformed. Plausibility, whether judged by the model that wrote the lesson or by a screen applied to the development data, did not predict out-of-sample value. Only a replay on a later, disjoint period did.

This is consistent with the selection principle in GEPA [1] and with the curation stage of ACE [23], and it suggests that for sequential trading the gate deserves at least as much design attention as

Table 6: Final-period performance across the eight comparison conditions. The last column reports the Sharpe ratio relative to the full-exposure base model.

Condition	Memories	Sharpe	Net return (%)	Maximum drawdown (%)	Sharpe vs. base (%)
Base model: full exposure	—	5.638	10,009.31	18.59	—
Base model: half exposure	—	5.556	959.25	9.61	−1.5
Model-disagreement rule*	—	5.631	9,649.80	18.44	−0.1
High-volatility rule*	—	5.201	2,736.72	16.90	−7.8
LLM without memory	0	5.003	4,292.64	17.49	−11.3
LLM with raw memory	20	5.254	1,019.82	10.89	−6.8
LLM with screened memory	5	4.996	1,530.85	9.61	−11.4
LLM with gate-selected memory	1	5.763	10,762.61	18.59	+2.2

Note: A single retained instruction can guide many different trading decisions; it does not merely record one trade. The validation gate was deliberately designed to select at most one lesson so that the impact of a single memory instruction could be measured. *Exploratory deterministic rules introduced after inspection of the original final-evaluation results.

Table 7: LLM reviewer actions and exposure statistics during final evaluation.

LLM condition	Keep	Halve	Flat	<i>E</i>
Without memory	473	159	0	0.589
Raw memory	96	536	0	0.382
Screened memory	166	466	0	0.414
Gate-selected memory	552	80	0	0.669

Note: Counts are over 632 decisions; FLAT was never selected. *E* is the mean absolute target exposure.

the reflection prompt. It also suggests a specific failure mode for memory that is accumulated without validation: lessons drafted from losing trades tend to counsel caution, caution compounds across a long horizon when the only available action is to shrink exposure, and the aggregate effect is a reviewer that trades a materially smaller book than the policy it is reviewing. The raw-memory condition kept 15.2% of proposals and returned a tenth of the base model’s profit.

7 Limitations

The evidence is thin in ways that bound every claim above.

One asset, one path. All results come from daily Ethereum data over a single historical trajectory. The six conditions share that trajectory, so their equity curves are not independent samples and the differences between them cannot be given a confidence interval or a *p*-value. We report point estimates only.

One lesson, selected from twenty. The gate retained a single instruction chosen for its validation Sharpe from five screened candidates, which were themselves drawn from twenty. Selecting the maximum of several noisy validation scores inflates the expected performance of the winner, a well-documented hazard in strategy backtesting that motivates multiple-testing corrections such as the deflated Sharpe ratio [2] and the haircuts of Harvey and Liu [6]. We do not apply such a correction here, and with one validation period we cannot separate that inflation from a real effect. The 2.2% final-period gain is small enough that it could plausibly be explained this way.

Replay realism. Sharpe ratios near 5.6 are far above what daily directional trading in this asset class normally produces, and they

reflect the replay rather than attainable performance. Funding and borrowing costs and interest on idle cash are not modeled; the original strategy’s take-profit and stop-loss logic is absent; and the proposal reconstruction feeds each PPO policy its previous proposal while setting the recent-return input to zero, which is an approximation of the observation the policy saw in training. The evaluation period also covers a strongly trending market, which rewards the full-exposure base model and leaves little room for a de-risking reviewer to add value.

Possible prior exposure to the evaluation period. The reviewer is a frozen commercial model evaluated on dates from January 2025 to September 2026. We cannot verify whether its training data covers that period, so we cannot rule out that it had some prior familiarity with the price history it was asked to judge; Glasserman and Lin [5] show that such look-ahead bias can inflate measured return predictability in LLM-generated financial signals. This affects the LLM conditions and not the scripted controls.

Restricted action set. The reviewer may only keep, halve, or flatten a proposal, and in practice never flattened. It cannot increase exposure, reverse a position, or alter the base signal, so the ceiling on what any memory can achieve in this setup is set by how much value is recoverable from declining risk alone.

8 Conclusion and Future Work

We asked whether a trained MLP–PPO trading system could be improved by a frozen LLM reviewer that accumulates experience as retained text rather than updated weights. On daily Ethereum data, a single instruction that survived a held-out validation gate raised the final-period Sharpe ratio from 5.638 to 5.763 and net return from 10,009% to 10,763% at unchanged drawdown, for roughly \$4 of API spending and no retraining. The same reviewer with no memory, with all twenty generated lessons, or with the five screened lessons performed 7–11% worse than the base system in Sharpe ratio.

Three extensions would make these results interpretable. First, repeating the study across several assets and multiple rolling development, validation, and evaluation windows would show whether the gate selects useful instructions more often than chance, which one path cannot. Second, allowing the gate to retain a small set of complementary instructions, rather than at most one, would test whether memories compose or whether their caution accumulates

as the raw-memory condition suggests. Third, widening the action set to permit increases in exposure would test whether the layer can contribute anything beyond selective de-risking.

References

- [1] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2025. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. [arXiv:2507.19457](#)
- [2] David H. Bailey and Marcos López de Prado. 2014. The Deflated Sharpe Ratio: Correcting for Selection Bias, Backtest Overfitting, and Non-Normality. *The Journal of Portfolio Management* 40, 5 (2014), 94–107.
- [3] Adam Darmanin and Vince Vella. 2025. Language Model Guided Reinforcement Learning in Quantitative Trading. [arXiv:2508.02366](#)
- [4] Yue Deng, Feng Bao, Youyong Kong, Zhiqian Ren, and Qionghai Dai. 2017. Deep Direct Reinforcement Learning for Financial Signal Representation and Trading. *IEEE Transactions on Neural Networks and Learning Systems* 28, 3 (2017), 653–664.
- [5] Paul Glasserman and Caden Lin. 2023. Assessing Look-Ahead Bias in Stock Return Predictions Generated by GPT Sentiment Analysis. [arXiv:2309.17322](#)
- [6] Campbell R. Harvey and Yan Liu. 2015. Backtesting. *The Journal of Portfolio Management* 42, 1 (2015), 13–28.
- [7] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2023. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. [arXiv:2310.03714](#)
- [8] Xiao-Yang Liu, Hongyang Yang, Qian Chen, Runjia Zhang, Liuqing Yang, Bowen Xiao, and Christina Dan Wang. 2020. FinRL: A Deep Reinforcement Learning Library for Automated Stock Trading in Quantitative Finance. [arXiv:2011.09607](#)
- [9] Marcos López de Prado. 2018. *Advances in Financial Machine Learning*. John Wiley & Sons, Hoboken, NJ.
- [10] Alejandro Lopez-Lira and Yuehua Tang. 2023. Can ChatGPT Forecast Stock Price Movements? Return Predictability and Large Language Models. [arXiv:2304.07619](#)
- [11] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. [arXiv:2303.17651](#)
- [12] John Moody and Matthew Saffell. 2001. Learning to Trade via Direct Reinforcement. *IEEE Transactions on Neural Networks* 12, 4 (2001), 875–889.
- [13] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as Operating Systems. [arXiv:2310.08560](#)
- [14] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. [arXiv:2304.03442](#)
- [15] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. [arXiv:1707.06347](#)
- [16] William F. Sharpe. 1994. The Sharpe Ratio. *The Journal of Portfolio Management* 21, 1 (1994), 49–58.
- [17] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. [arXiv:2303.11366](#)
- [18] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. [arXiv:2305.16291](#)
- [19] Yijia Xiao, Edward Sun, Di Luo, and Wei Wang. 2024. TradingAgents: Multi-Agents LLM Financial Trading Framework. [arXiv:2412.20138](#)
- [20] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2023. Large Language Models as Optimizers. [arXiv:2309.03409](#)
- [21] Yangyang Yu, Haohang Li, Zhi Chen, Yuechen Jiang, Yang Li, Denghui Zhang, Rong Liu, Jordan W. Suchow, and Khaldoun Khashanah. 2023. FinMem: A Performance-Enhanced LLM Trading Agent with Layered Memory and Character Design. [arXiv:2311.13743](#)
- [22] Yangyang Yu, Zhiyuan Yao, Haohang Li, Zhiyang Deng, Yupeng Cao, Zhi Chen, Jordan W. Suchow, Rong Liu, Zhenyu Cui, Zhaozhao Xu, Denghui Zhang, Koduvayur Subbalakshmi, Guojun Xiong, Yueru He, Jimin Huang, Dong Li, and Qianqian Xie. 2024. FinCon: A Synthesized LLM Multi-Agent System with Conceptual Verbal Reinforcement for Enhanced Financial Decision Making. [arXiv:2407.06567](#)
- [23] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. 2025. Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models. [arXiv:2510.04618](#)
- [24] Wentao Zhang, Lingxuan Zhao, Haochong Xia, Shuo Sun, Jiaze Sun, Molei Qin, Xinyi Li, Yuqing Zhao, Yilei Zhao, Xinyu Cai, Longtao Zheng, Xinrun Wang, and Bo An. 2024. A Multimodal Foundation Agent for Financial Trading: Tool-Augmented, Diversified, and Generalist. [arXiv:2402.18485](#)
- [25] Zihao Zhang, Stefan Zohren, and Stephen Roberts. 2019. Deep Reinforcement Learning for Trading. [arXiv:1911.10107](#)
- [26] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2023. ExpeL: LLM Agents Are Experiential Learners. [arXiv:2308.10144](#)
- [27] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. 2022. Large Language Models Are Human-Level Prompt Engineers. [arXiv:2211.01910](#)